# Full Stack Development with MERN

# Project Documentation

## 1. Introduction

- **Project Title:** ShopEZ: E-Commerce Application
- **Name: Mudit Manglik**

## 2. Project Overview

- **Purpose:** ShopEZ is designed to bridge the gap between static frontend shopping cart mockups and fully integrated, database-backed enterprise applications. The project replaces transient local state variables and fake API fetches with persistent database operations, secure user/administrator authentication, and transactional order records.

- **Features:**

    - Interactive Catalog: Category and gender-filtered fashion listings featuring discounts and descriptions.

    - Customer Reviews: Product-specific ratings and comment sections populated from seeded collections.

    - Persistent Database Cart: Syncs automatically with the backend upon customer login.

    - Direct Checkout: Form-validated shipping entries and purchase history tables.

    - Admin Dashboard: Control panel enabling product CRUD, invoice auditing, and user lists.

- .

## 3. Architecture

- **Frontend:** A React.js (v19) Single Page Application bundled with Vite, styled with Tailwind CSS, and powered by:

    - **React Router DOM (v7):** Configures declarative client-side route paths.

    - **Context API:** Manages global state variables (e.g. login tokens, cart counts, API helpers).

- **Backend:** An Express.js and Node.js REST API organized around the Model-View-Controller (MVC) design pattern:

    - Model Layer: Handles Mongoose schemas and data formatting.
    - Controller Layer: Contains handlers running query actions and returning JSON.
    - View/Routing Layer: Maps HTTP endpoints (GET, POST, PUT, DELETE) to controllers.

- **Database:** MongoDB document-based storage. Collections are modeled using **Mongoose ODM**:
  - Users: Tightly manages passwords and access roles (USER/ADMIN).
  - Products: Stores product inventory and nested arrays of customer review subdocuments.
  - Cart: Connects shopping quantities and sizes to user ObjectIds.
  - Orders: Records delivery information and invoice transaction details.

## 4. Setup Instructions

- **Prerequisites:**
  - Node.js (v18.0.0 or higher)
  - npm (v9.0.0 or higher)
  - MongoDB Community Server running locally on port 27017

- **Installation:**

1. Navigate to the project root directory:

```bash
cd C:/Users/Mudit/OneDrive/Desktop/SmartBridge/Shopping-Cart-main
```

2. Setup Backend Server environment: Create a file named `.env` inside the `server/` directory:

```env
PORT=5000
MONGO_URI=mongodb://127.0.0.1:27017/shopez
JWT_SECRET=shopezsupersecretjwtkey123!
```

3. Install dependencies across both client and server:

```bash
cd client && npm install
cd ../server && npm install
```

## 5. Folder Structure

- **Client:**

client/

├── src/

│ ├── assets/ # Images, fonts, and stylesheets

│ ├── components/ # Reusable widgets (NavBar, CartItem, ProductCard, PriceDetails)

│ ├── context/ # ShopContext global state API

│ ├── pages/ # Screen views (Home, Shop, Checkout, Orders, AdminDashboard, Login, Signup)

│ ├── routes/ # Declarative router pathways

│ ├── App.jsx # Context provider initialization

│ └── main.jsx # Client entrypoint

├── index.html # HTML container

├── vite.config.js # Vite build configurations

└── package.json # Client dependencies

- **Server:**

server/
├── config/ # Database Mongoose connector
├── middleware/ # JWT and role-based protection
├── db/ # Core Models (User.js, Admin.js)
├── models/ # Schema blueprints (Product.js, Cart.js, Order.js)
├── controllers/ # MVC handler logic (auth, products, cart, orders)
├── routes/ # Express endpoint paths (auth, products, cart, orders)
├── seed.js # Database population script
├── server.js # Express API server core
└── package.json # Server dependencies

## 6. Running the Application

- Commands to start the frontend and backend servers locally.
  - **Frontend:**

```
cd client
npm run dev
```

- **Backend:**

```
cd server
npm run seed
npm run dev
```

## 7. API Documentation

### 7.1 Authentication (/api/auth)

- POST /register
  - **Body:** { "username": "testuser", "email": "test@example.com", "password": "password123" }
  - **Response:** 201 Created returning user model and { "token": "JWT_STRING" }.
- POST /login
  - **Body:** { "email": "customer@shopez.com", "password": "password123" }
  - **Response:** { "_id": "ID", "username": "customer123", "token": "JWT" }.

### 7.2 Products (/api/products)

- GET /: Lists all products. Returns { "products": [...] }.
- GET /category/:slug: Lists products matching category slug.
- GET /:id: Detailed single product information with reviews.
- POST / *(Protected/Admin)*: Adds a product.
- DELETE /:id *(Protected/Admin)*: Deletes a product.

### 7.3 Cart (/api/cart)

- GET / *(Protected)*: Lists user cart items.
- POST / *(Protected)*: Adds/updates item size and quantity in database.

### 7.4 Orders (/api/orders)

- POST / *(Protected)*: Submits cart items, creates orders, and flattens them into the database.
- GET /my-orders *(Protected)*: Lists past invoices of the active user.

## 8. Authentication

**Security Strategy:**

- **Password Cryptography:** Users' plain-text passwords are encrypted using bcryptjs with a salt factor of 10 prior to writing to the database.

- **Session Authorization:** Login requests check the hashed entry, sign a JSON Web Token (JWT) with the user ID, and send it back to the client.

- **Header Verification:** Protected pages send the JWT inside the header: Authorization: Bearer <TOKEN>. Express middleware intercepts, verifies, and appends the user context req.user for matching endpoints.

## 9. User Interface

Sleek Client Features:

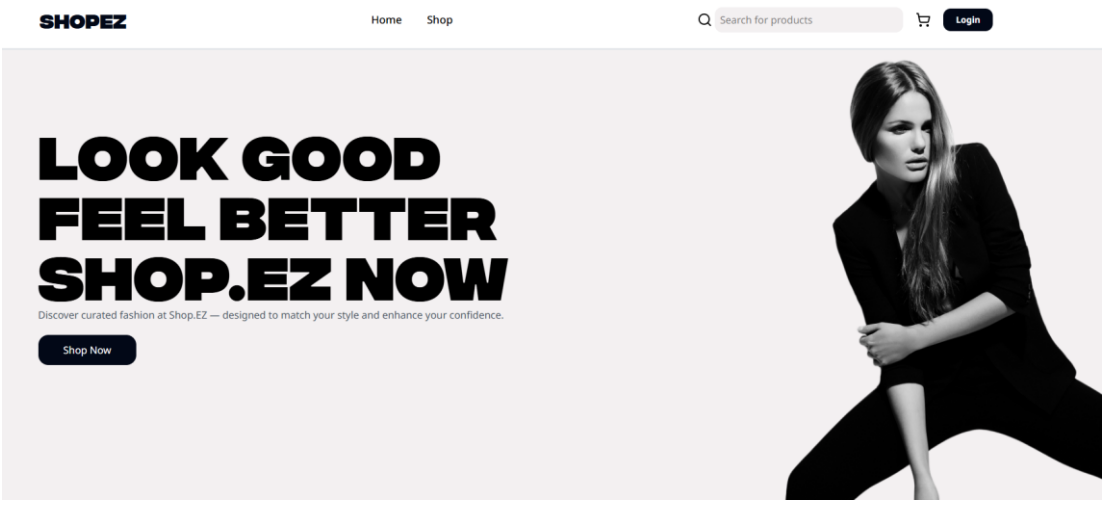
- Navigation Bar: Displays cart quantities and dynamic links (e.g., Admin and Orders buttons based on role permissions).
- Catalogue Page: Supports category sorting and includes quick-action Shop Now buttons.
- Product Details: Prominently lists custom sizing blocks, a secondary Shop Now buy option, and nested client feedback reviews.
- Checkout Form: Form-validated shipping page capturing contact details and payment options.

## 10. Testing

Testing Strategy:

- Frontend Verification: Unit tests configured in Vitest and React Testing Library verify that UI components render correctly.
- Compilation Checks: The Vite bundler acts as a compilation check. Running npm run build verifies import pathways and layout dependencies.
- Database Validation: Seeder outputs and Express request logs verify correct MongoDB schema reads/writes.

## 11. Screenshots or Demo



## FRAGRANCES

Calvin Klein CK One
$49.99
Shop Now

Chanel Coco Noir Eau De
$129.99
Shop Now


Dior J'adore
$89.99
Shop Now

Dolce Shine Eau de
$69.99
Shop Now




Gucci Bloom Eau de
$79.99
Shop Now

View All

**SHOP.EZ**

Discover curated fashion at Shop.EZ — designed to match your style and enhance your confidence.

| COMPANY | HELP | FAQ | RESOURCES |
|---|---|---|---|
| About | Customer Support | Account | Free Ebooks |
| Careers | Delivery Details | Manage Deliveries | Development Tutorial |
| Contact | Terms and Conditions | Orders | How to - Blog |
| Features | Privacy Policy | Payments | Youtube Playlist |

## ALL PRODUCTS

**Categories**

All
Mens Shirts
Mens Shoes
Womens Dresses
Womens Shoes
Womens Bags
Tops
Fragrances

Classic White Linen Shirt
$45
Shop Now




Casual Blue Denim Shirt
$55
Shop Now

Leather Oxford Shoes
$120
Shop Now

Summer Floral Maxi Dress
$75
Shop Now

Elegant Black Evening Gown
$150
Shop Now

Designer Leather Handbag
$210
Shop Now

**Categories**

All
Mens Shirts
Mens Shoes
Womens Dresses
Womens Shoes
Womens Bags
Tops
Fragrances

## WOMENS DRESSES

Summer Floral Maxi Dress
$75
Shop Now

Elegant Black Evening Gown
$150
Shop Now

**SHOPEZ** Home Shop Orders Search for products Hi, customer123

## YOUR CART

Designer Leather Handbag
**$210**
- 1 +

Chanel Coco Noir Eau De
**$129.99**
- 1 +

Elegant Black Evening Gown
**$150**
- 1 +

**Order Summary**

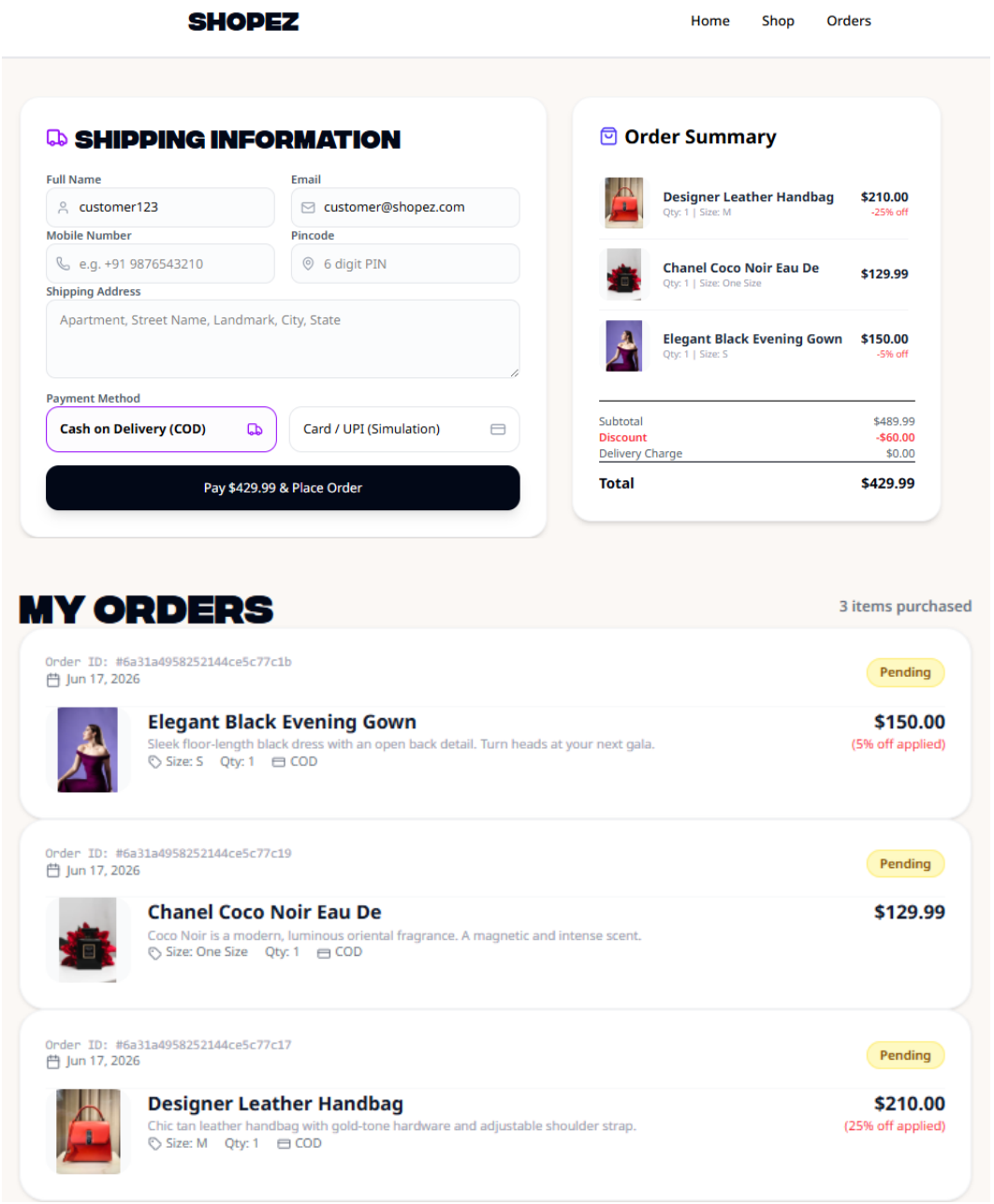
| | |
|---|---|
| Subtotal | $489.99 |
| Discount | -$60.00 |
| Delivery Fee | $0.00 |
| **Total** | **$429.99** |

PROCEED TO CHECKOUT

**SHOP.EZ**

Discover curated fashion at Shop.EZ — designed to match your style and enhance your confidence.

| COMPANY | HELP | FAQ | RESOURCES |
|---|---|---|---|
| About | Customer Support | Account | Free Ebooks |
| Careers | Delivery Details | Manage Deliveries | Development Tutorial |
| Contact | Terms and Conditions | Orders | How to - Blog |
| Features | Privacy Policy | Payments | Youtube Playlist |




## 12. Known Issues

 **Simulated Payments:** The card/UPI billing process does not call an active production gateway; it performs a secure API simulation that logs completed COD or CARD states in the database.

 **Localhost Origin:** Cross-Origin Resource Sharing (CORS) limits allow standard requests from localhost; deploying to production requires updating server CORS domains.

## 13. Future Enhancements

 **Payment Integration:** Incorporate Stripe / Razorpay for live card transactions.

 **Email Notifications:** Trigger automated invoice confirmations using Nodemailer.

 **Dynamic Search:** Implement fuzzy search query matches on database products.